

Dijkstra's Algorithm based Common Questions

Last Updated : 06 Oct, 2025

Dijkstra's Algorithm was introduced by Dutch computer scientist **Edsger W. Dijkstra in 1956**. It is one of the most popular algorithms in graph theory, used to find the **shortest path** from a single source to all other vertices in a graph with non-negative edge weights.

The algorithm works by maintaining a set of visited and unvisited vertices. Starting from the source it repeatedly picks the vertex with the smallest tentative distance, updates the distances of its neighbours and continues until all vertices are processed or the destination is reached.

Can Dijkstra's Algorithm work on both Directed and Undirected graphs?

Yes, Dijkstra's algorithm can work on both directed graphs and undirected graphs as this algorithm is designed to work on any type of graph as long as it meets the requirements of having non-negative edge weights and being connected.

- **In a directed graph**, each edge has a direction, indicating the direction of travel between the vertices connected by the edge. In this case, the algorithm follows the direction of the edges when searching for the shortest path.
- **In an undirected graph**, the edges have no direction, and the algorithm can traverse both forward and backward along the edges when searching for the shortest path.

Algorithm Steps

1. Mark the source node with a current distance of 0 and the rest with infinity.
2. Set the non-visited node with the smallest current distance as the current node.
3. For each neighbor, N of the current node adds the current distance of the adjacent node with the weight of the edge connecting 0->1. If it is smaller

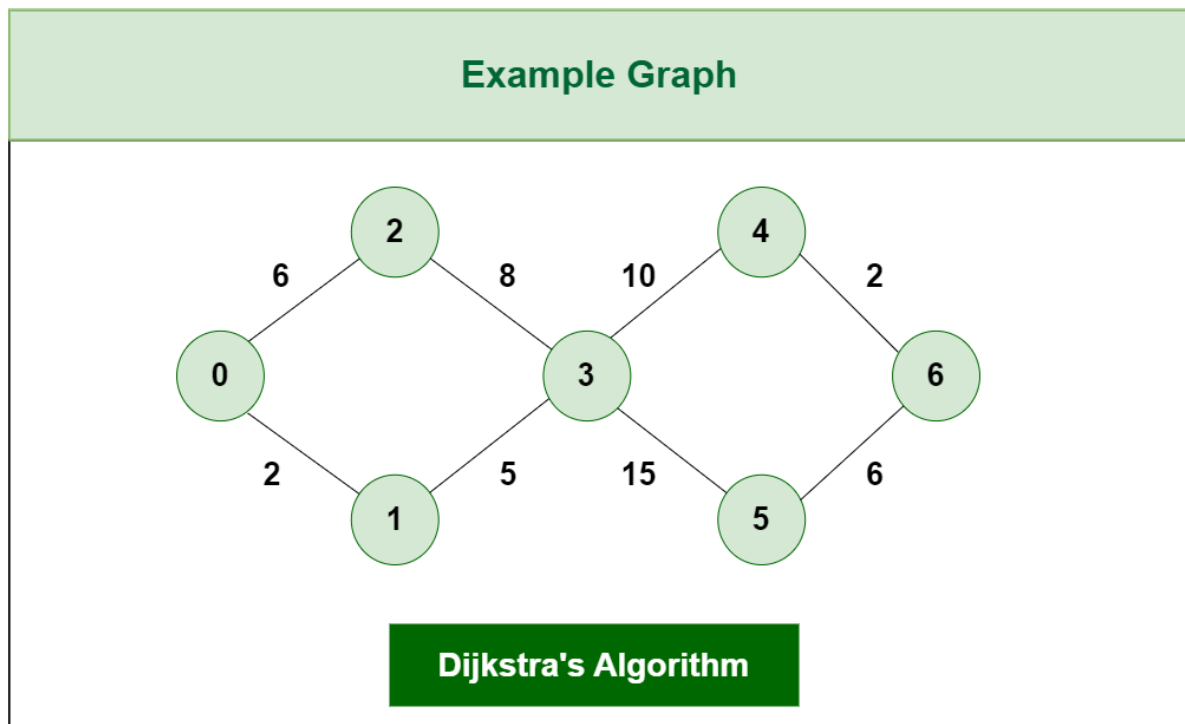
- than the current distance of Node, set it as the new current distance of N.
4. Mark the current node 1 as visited.
 5. Go to step 2 if there are any nodes are unvisited.

How does Dijkstra's Algorithm works?

Let's see how Dijkstra's Algorithm works with an example given below:

Dijkstra's Algorithm will generate the shortest path from Node 0 to all other Nodes in the graph.

Consider the below graph:



Dijkstra's Algorithm

The algorithm will generate the shortest path from node 0 to all the other nodes in the graph.

For this graph, we will assume that the weight of the edges represents the distance between two nodes.

Initially we have:

- The Distance from the source node to itself is 0. In this example the source node is 0.
- The distance from the source node to all other node is unknown so we mark all of them as infinity.

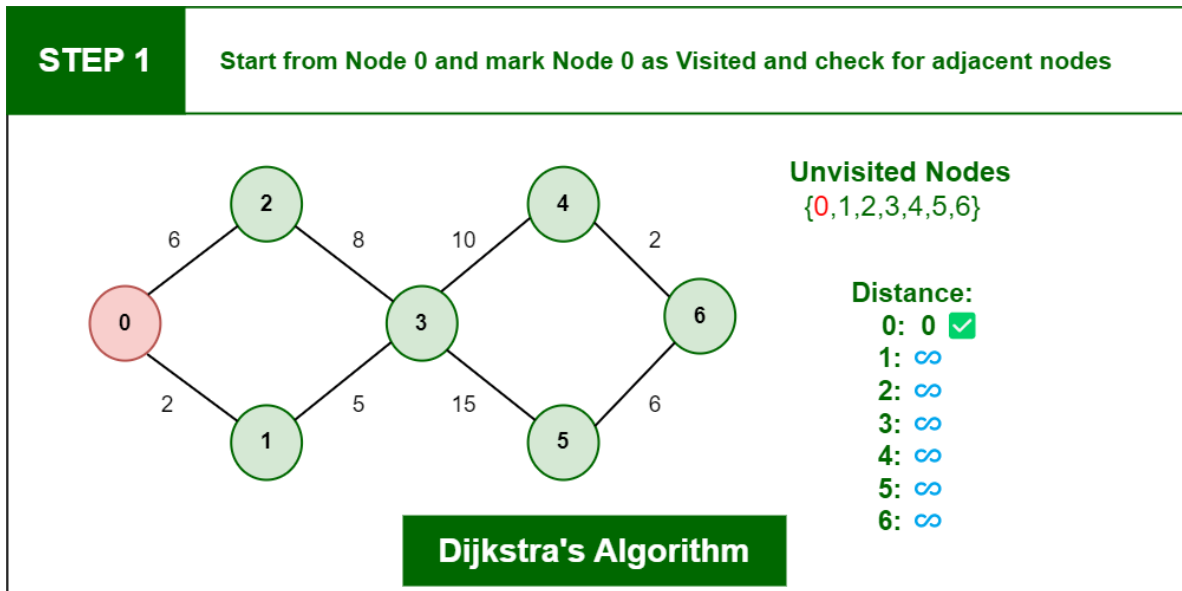
Example: 0 -> 0, 1-> ∞ , 2-> ∞ , 3-> ∞ , 4-> ∞ , 5-> ∞ , 6-> ∞ .

- we'll also have an array of unvisited elements that will keep track of

unvisited or unmarked Nodes.

- Algorithm will complete when all the nodes marked as visited and the distance between them added to the path. **Unvisited Nodes:- 0 1 2 3 4 5 6.**

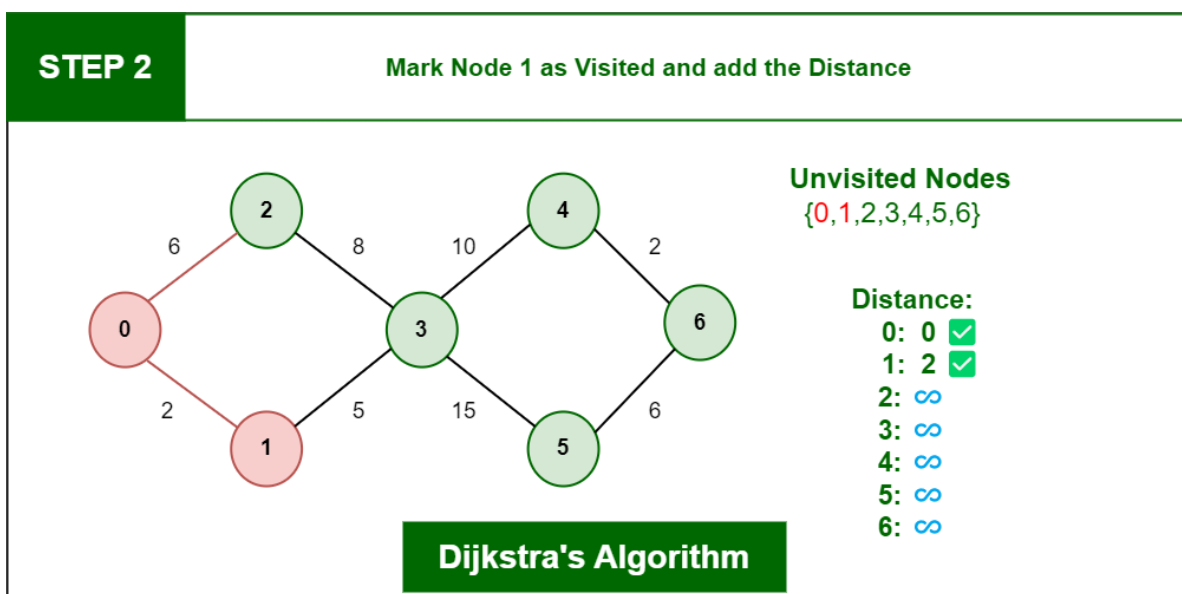
Step 1: Start from Node 0 and mark Node as visited as you can check in below image visited Node is marked red.



Dijkstra's Algorithm

Step 2: Check for adjacent Nodes, Now we have to choices (Either choose Node1 with distance 2 or either choose Node 2 with distance 6) and choose Node with minimum distance. In this step **Node 1** is Minimum distance adjacent Node, so marked it as visited and add up the distance.

Distance: Node 0 -> Node 1 = 2

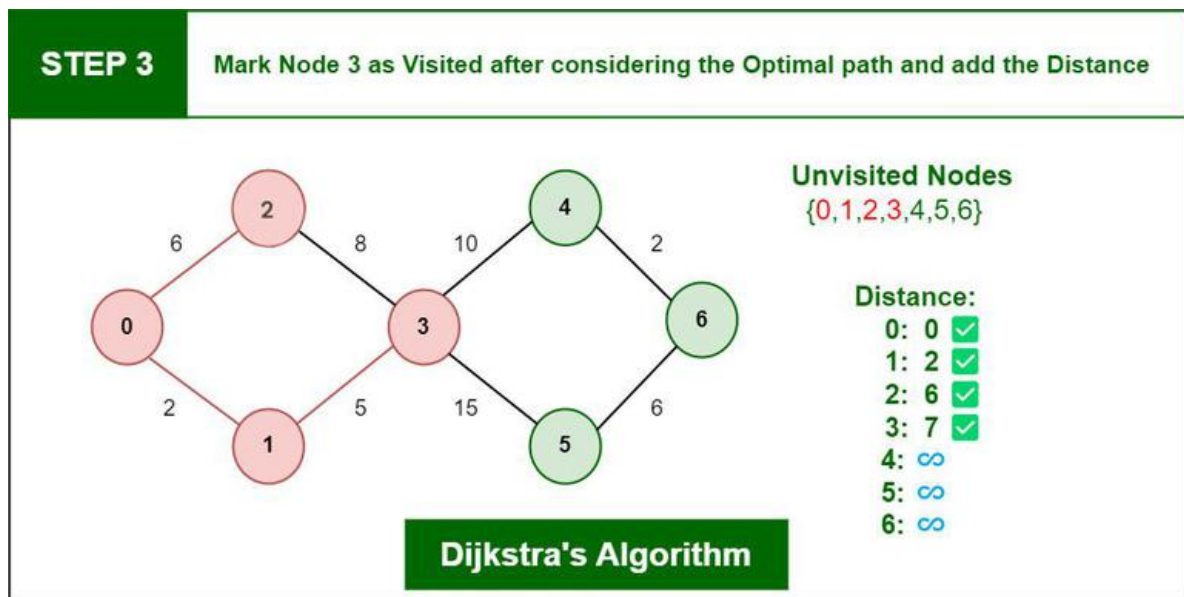


Dijkstra's Algorithm

Step 3: Then Move Forward and check for adjacent Node which is Node 3, so

marked it as visited and add up the distance, Now the distance will be:

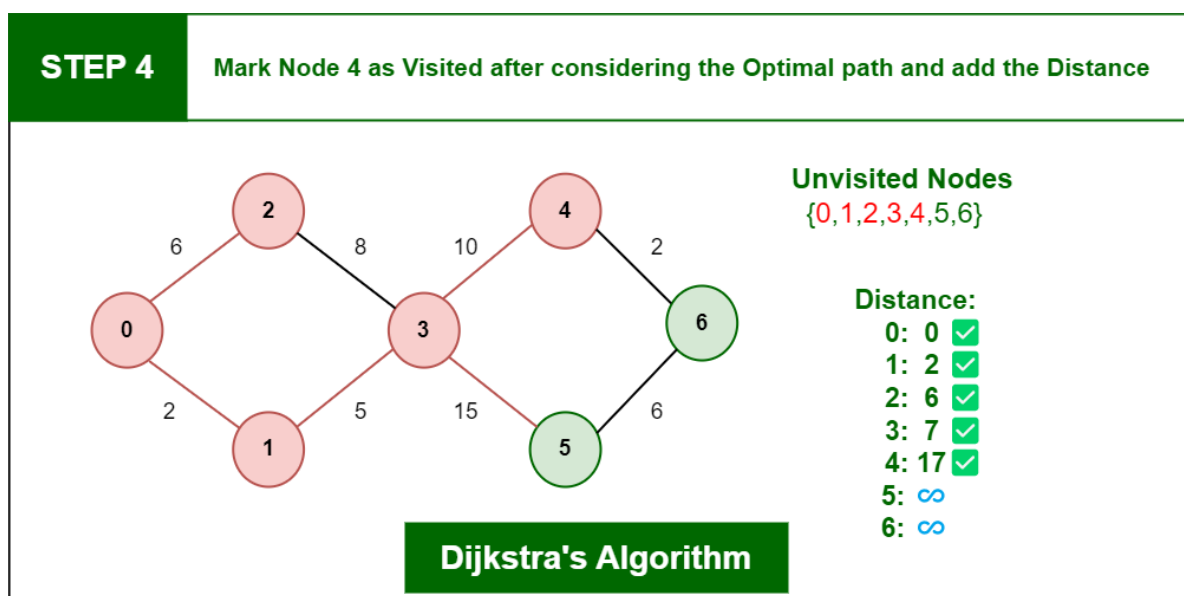
Distance: Node 0 -> Node 1 -> Node 3 = 2 + 5 = 7



Dijkstra's Algorithm

Step 4: Again we have two choices for adjacent Nodes (Either we can choose Node 4 with distance 10 or either we can choose Node 5 with distance 15) so choose Node with minimum distance. In this step **Node 4** is Minimum distance adjacent Node, so marked it as visited and add up the distance.

Distance: Node 0 -> Node 1 -> Node 3 -> Node 4 = 2 + 5 + 10 = 17

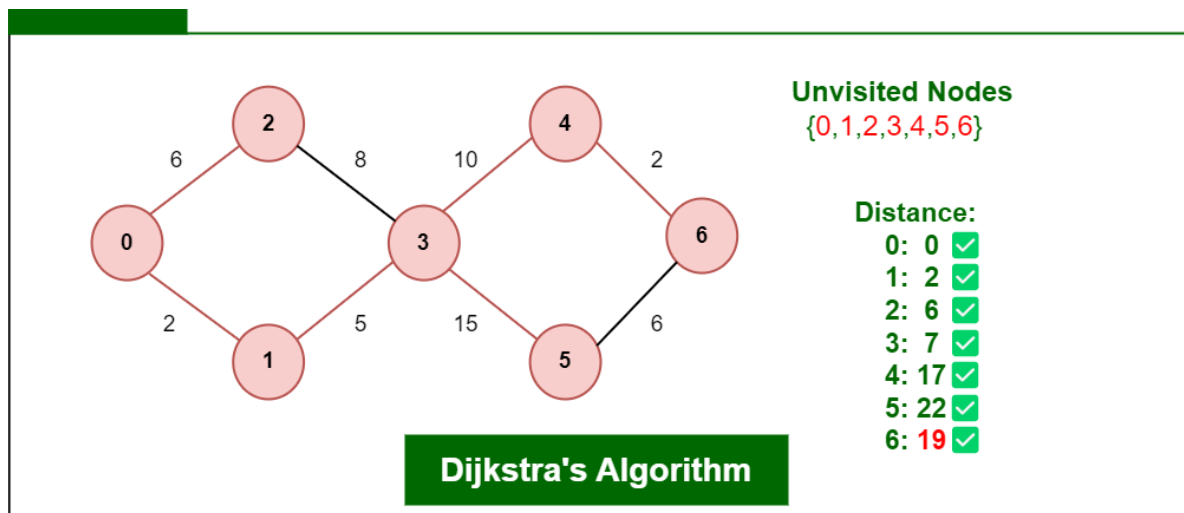


Dijkstra's Algorithm

Step 5: Again, Move Forward and check for adjacent Node which is **Node 6**, so marked it as visited and add up the distance, Now the distance will be:

Distance: Node 0 -> Node 1 -> Node 3 -> Node 4 -> Node 6 = 2 + 5 + 10 + 2 = 19





Dijkstra's Algorithm

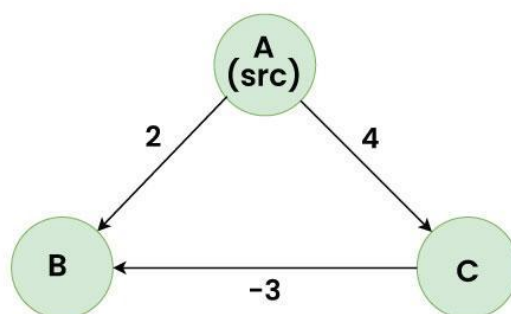
So, the Shortest Distance from the Source Vertex is 19 which is optimal one

For an implementation of Dijkstra's algorithm, refer to this [article](#).

Why Dijkstra's Algorithms fails for the Graphs having Negative Edges ?

The problem with negative weights arises from the fact that Dijkstra's algorithm assumes that once a node is added to the set of visited nodes, its distance is finalized and will not change. However, in the presence of negative weights, this assumption can lead to incorrect results.

Consider the following graph for the example:



Shortest Distance A to B using Dijkstra = 2

Actual Shortest Distance A to B = $4 - 3 = 1$

Failure of Dijkstra in case of negative edges.



In the above graph, A is the source node, among the edges A to B and A to C, A to B is the smaller weight and Dijkstra assigns the shortest distance of B as 2, but because of existence of a negative edge from C to B, the actual

shortest distance reduces to 1 which Dijkstra fails to detect.

Note: We use [Bellman Ford's Shortest path algorithm](#) in case we have negative edges in the graph.

Dijkstra's Algorithms vs Bellman-Ford Algorithm

Dijkstra's algorithm and [Bellman-Ford algorithm](#) are both used to find the shortest path in a weighted graph, but they have some key differences. Here are the main differences between Dijkstra's algorithm and Bellman-Ford algorithm:

Feature:	Dijkstra's	Bellman Ford
Optimization	optimized for finding the shortest path between a single source node and all other nodes in a graph with non-negative edge weights	Bellman-Ford algorithm is optimized for finding the shortest path between a single source node and all other nodes in a graph with negative edge weights.
Relaxation	Dijkstra's algorithm uses a greedy approach where it chooses the node with the smallest distance and updates its neighbors	the Bellman-Ford algorithm relaxes all edges in each iteration, updating the distance of each node by considering all possible paths to that node
Time Complexity	Dijkstra's algorithm has a time complexity of $O(V^2)$ for a dense graph and $O(E \log V)$ for a sparse graph, where V is the number of vertices and E is the number of edges in the graph.	Bellman-Ford algorithm has a time complexity of $O(VE)$, where V is the number of vertices and E is the number of edges in the graph.
Negative Weights	Dijkstra's algorithm does not work with graphs that have negative edge weights, as it assumes	Bellman-Ford algorithm can handle negative edge weights and can detect negative-weight

Feature:	Dijkstra's	Bellman Ford
	that all edge weights are non-negative.	cycles in the graph.

Dijkstra's Algorithm vs Floyd-Warshall Algorithm

Dijkstra's algorithm and [Floyd-Warshall algorithm](#) are both used to find the shortest path in a weighted graph, but they have some key differences. Here are the main differences between Dijkstra's algorithm and Floyd-Warshall algorithm:

Feature:	Dijkstra's	Floyd-Warshall Algorithm
Optimization	Optimized for finding the shortest path between a single source node and all other nodes in a graph with non-negative edge weights	Floyd-Warshall algorithm is optimized for finding the shortest path between all pairs of nodes in a graph.
Technique	Dijkstra's algorithm is a single-source shortest path algorithm that uses a greedy approach and calculates the shortest path from the source node to all other nodes in the graph.	Floyd-Warshall algorithm, on the other hand, is an all-pairs shortest path algorithm that uses dynamic programming to calculate the shortest path between all pairs of nodes in the graph.
Time Complexity	Dijkstra's algorithm has a time complexity of $O(V^2)$ for a dense graph and $O(E \log V)$ for a sparse graph, where V is the number of vertices and E is the number of edges in the graph.	Floyd-Warshall algorithm, on the other hand, is an all-pairs shortest path algorithm that uses dynamic programming to calculate the shortest path between all pairs of

Feature:	Dijkstra's	Floyd-Warshall Algorithm
		nodes in the graph.
Negative Weights	Dijkstra's algorithm does not work with graphs that have negative edge weights, as it assumes that all edge weights are non-negative.	Floyd-Warshall algorithm, on the other hand, is an all-pairs shortest path algorithm that uses dynamic programming to calculate the shortest path between all pairs of nodes in the graph.

Dijkstra's Algorithm vs A* Algorithm

Dijkstra's algorithm and [A* algorithm](#) are both used to find the shortest path in a weighted graph, but they have some key differences. Here are the main differences between Dijkstra's algorithm and A* algorithm:

Feature:		A* Algorithm
Search Technique	Optimized for finding the shortest path between a single source node and all other nodes in a graph with non-negative edge weights	A* algorithm is an informed search algorithm that uses a heuristic function to guide the search towards the goal node.
Heuristic Function	Dijkstra's algorithm, does not use any heuristic function and considers all the nodes in the graph.	A* algorithm uses a heuristic function that estimates the distance from the current node to the goal node. This heuristic function is admissible, meaning that it never overestimates the actual distance to the goal node

Feature:		A* Algorithm
Time Complexity	Dijkstra's algorithm has a time complexity of $O(V^2)$ for a dense graph and $O(E \log V)$ for a sparse graph, where V is the number of vertices and E is the number of edges in the graph.	The time complexity of A* algorithm depends on the quality of the heuristic function.
Application	Dijkstra's algorithm is used in many applications such as routing algorithms, GPS navigation systems, and network analysis	. A* algorithm is commonly used in pathfinding and graph traversal problems, such as video games, robotics, and planning algorithms.

Also Check:

1. [Dijkstra's shortest path algorithm | Greedy Algo-7](#)
2. [Dijkstra's Algorithm for Adjacency List Representation | Greedy Algo-8](#)
3. [Dijkstra's Algorithm – Priority Queue and Array Implementation](#)
4. [Dijkstra's shortest path algorithm using set in STL](#)
5. [Dijkstra's shortest path algorithm using STL in C++](#)
6. [Dijkstra's Shortest Path Algorithm using priority_queue of STL](#)
7. [Dijkstra's shortest path algorithm using matrix in C++](#)
8. [Dijkstra's Algorithm for Single Source Shortest Path in a DAG](#)
9. [Dijkstra's Algorithm using Fibonacci Heap](#)
10. [Dijkstra's shortest path algorithm for directed graph with negative weights](#)
11. [Printing Paths in Dijkstra's Shortest Path Algorithm](#)
12. [Dijkstra's shortest path algorithm with priority queue in Java](#)
13. [Dijkstra's shortest path algorithm using adjacency matrix in Java](#)
14. [Dijkstra's Algorithm using Adjacency List in Python](#)
15. [Dijkstra's Algorithm using PriorityQueue in Python](#)
16. [Dijkstra's Algorithm using heapq module in Python](#)
17. [Dijkstra's Algorithm using dictionary and priority queue in Python](#)